📄 [부글부글 프로젝트 사양 설계서]

목차

1. 프로젝트 개요
2. 개발범위 설정
3. 기능 명세
4. 게임 Flow
5. 시스템 구조
6. 데이터 구조
7. 인터페이스 정의
8. 역할 분담
9. 프로젝트 구조
10. 개발 일정
11. Git 규칙
12. 완료 기준
13. 리스크 및 대응
14. AI 사용 규칙

---

1. 프로젝트 개요

- 프로젝트명: 부글부글
- 개발 기간: 2026.03.25 ~ 2026.03.31
- 개발 인원: 4명
  - 김진수, 윤성민, 인정원, 최이레
- 개발 환경:
  - Language: C
  - Library: Allegro5
  - IDE: Visual Studio
  - 형상 관리: GIT
  - 작업 공유: Notion
- 목표: 협업능력, C 코드 분석능력 향상

---

2. 개발 범위 설정 (제약)

2.1 반드시 필요한 기능

■ 충돌 정책

- Player vs MAP
- Player vs Enemy

- Bubble vs Enemy
- 충돌 방식: AABB(박스 처리)
- 충돌 처리: 충돌 시 위치 보정 및 상태 변화

■ 좌표계 / 단위

- 기준: 좌측 상단 (0,0)
- 단위: 알잘딱
- Y축: 아래 방향 증가

■ 입력 처리 방식

- Polling 방식 (프레임당 1회)
- 동시 입력 허용
- 입력 정책:
  - 이동: Key Hold
  - 공격: Key Down

■ Stage

- Stage1
  - 움직이기만 하는 적
    - 6마리
- Stage2
  - 날아가는 공격도 하는 적
    - 6마리
- Stage3
  - 움직이기만 하는 적
    - 5마리
  - 날아가는 공격도 하는 적
    - 3마리
  - 보스(시간남으면 구현)

■ Player Stat

- 이동
  - 현재 좌표에 dY,dX를 통해서 선형 증가
  - 동시 입력 허용
  - 하향 점프(나중에 생각)
- 공격
  - 공격 딜레이: 1초에 한번
  - 바라보는 방향으로 발사

    - 일정 거리 이동 후 멈추고 떠오르게
    - 벽에 닿으면 바로 거품 생성
    - 일정 시간 지나면 사라짐 → 적이 갇힌 거품이라면 적 속도 증가
    - 거품에 갇힌 적은 플레이어가 이동해서 접촉하면 사망
- 무적 시간
    - 3초

■ 적 STAT

- 움적
    - 이동속도: Player와 같음
        - 특정 시간 경과 시 빨라짐
    - 점프: 가능
    - 공격
        - 닿으면 플레이어 사망
- 날적
    - 이동속도: Player와 같음
        - 특정 시간 경과 시 빨라짐
    - 점프: 가능
    - 공격
        - 닿으면 플레이어 사망
        - 투사체
            - 플레이어 방향으로 발사
            - 발사 Delay: 1~3초 랜덤
- 보스(추가 예정)

■ 종료

- 종료 조건
    - ESC
    - 적 모두 사망
    - 플레이어 사망
- 종료 시 이름입력 이후 랭킹 출력(내림차 순, TOP10)
- Continue / Restart

■ 렌더링

- 맵
    - 일단 맵에 찍어내고 충돌은 Collision 판정
        - Tile Object 필요
- UI

    - HUD
        - 점수
            - 시간 기준 + 남은 목숨
        - 목숨
            - 3개
        - 남은 시간
    - Continue/Restart
    - 랭킹
    - 게임 시작 화면
- Player / Enemy
    - State 따라서 Sprite 변경
    - 가만히 있어도 Sprite 변경 됨

■ Object

- 종류
    - Tile
        - 맵 구성 요소
    - Player
    - Enemy
    - 날적 투사체
    - 버블
- 요소
    - Collision
        - Width/Height
    - 좌표 변경 가능 여부
        - Static / Dynamic
    - 좌표 값(float)
    - dX,dY
    - Tag

■ Update 순서

```
1. input
2. player update
3. enemy update
4. physics apply
5. collision resolve
6. state update
7. render
```

2.2 있으면 좋은 기능

- 사운드 효과
- 간단한 UI 애니메이션
- 보스
- 공격 확산

2.3 제외 기능

- 복잡한 충돌 판정
- 멀티플레이
- 고급 AI

---

3. 기능 명세

입력 처리

- 입력: 입력 이벤트 Key 값
- 동작: 받은 Key값을 인덱스로 사용해서 배열에 저장
- 출력: 없음(다른 동작에서 활용)
- 제약: Flag이므로 아무리 많은 동일입력이 들어와도 최초 한번만 Flag

플레이어 이동

- 입력: 방향키
- 동작: 좌우 이동
- 출력: 위치 변화
- 제약: Collision 판정으로 이동 제한

플레이어 공격 (Bubble)

- 입력: 스페이스
- 동작: 거품 발사
- 출력: 적과 충돌 시 상태 변화
- 제약: 1초 Delay, Collision 닿으면 상호작용 후 소멸

플레이어 점프

- 입력: 위쪽 방향키
- 동작: dY에 의해서 선형 증가
- 출력: 위치 변화
- 제약: Collision 판정으로 이동 제한

날아가는 적 공격

- 입력: Random 시간 체크

- 동작: 날아가는 Object 사출
- 출력: Player Tag Object와 충돌 시 Player의 목숨 하나 깎음
- 제약: Map 크기 넘어가면 삭제, Delay는 3~5초 랜덤

적 AI 움직임

- 이레님 위임

충돌 판정

- 입력: 모든 Dynamic Object의 Collision
- 동작: Collision 닿는 모든 Object에 대해 Tag에 따라서 상태 변화
- 출력: Object간의 상호작용에 따른 상태 변화
- 제약: Tag 기준 동작

랭킹 출력

- 입력: 게임 종료 시 현재 점수 값
- 동작: 랭킹데이터 비교 후 크면 넣어줌
- 출력: 랭킹보드 렌더링
- 제약: 10위 초과 시 랭킹보드 반영 X 현재 점수만 렌더링

---

4. 게임 Flow (FSM)

```
Start
 ↓
Game Play
 ↓
Restart / Main
```

---

5. 시스템 구조

5.1 기본 구조

```
while (running) {
    Input();
    UpdateGame();
    RenderGame();
}
```

5.2 모듈 구조

```
- object
        - tile
    - player
    - enemy
```

```
    - bubble
    - 적 투사체
```

---

6. 데이터 구조

```
typedef struct {
    int y;
    int x;
} stPOSITION;

enum {
    eDIR_LOOK_UP = 0,
    eDIR_LOOK_DOWN,
    eDIR_LOOK_LEFT,
    eDIR_LOOK_RIGHT,
    eDIR_LOOK_MAX
} eDIR_LOOK;

typedef struct {
        int height;
        int width;
} stCOLLISION;

typedef struct {
        stPOSITION pos;
        stPOSITION speed;

    stCOLLISION collision; // 정원님 맡김
    sprite * sprite; // 동적 할당 받아 사용
    eTAG tag;
    bool is_static; // 움직일수 있는가
    eDIR_LOOK dir_look;
} stOBJECT;

typedef struct {
        stOBJECT obj;
        ePLAYER_STATE state;
        int heart;
        int 무적시간;
        int 마지막 공격시점;
} stPLAYER;

typedef enum {
        ePLAYER_STATE_IDLE = 0,
        ePLAYER_STATE_MOVE,
        ePLAYER_STATE_JUMP,
        ePLAYER_STATE_ATTACK,
        ePLAYER_STATE_DEAD,
        ePLAYER_STATE_MAX
} ePLAYER_STATE;
```

```
// 나머진 각자 역할에 맞춰 하위 구현, Player, Enemy

typedef enum {
        eENEMY_STATE_IDLE = 0,
    eENEMY_STATE_MOVE,      /* 좌우로 이동하며 지형을 탐색하는 상태 */
    eENEMY_STATE_JUMP,      /* 플레이어의 위치를 따라가기 위해 점프하는 상태 */
    eENEMY_STATE_ATTACK,    /* 원거리 공격(Mighta의 돌 던지기 등)을 수행하는 상태 */
    eENEMY_STATE_TRAPPED,   /* 거품에 갇혀 무력화된 상태 */
      eENEMY_STATE_DEAD
} eENEMY_STATE;

typedef struct {
        stOBJECT obj;
    AIState current_state;
    int state_timer;      /* 현재 상태 유지 시간 [30, 31] */
    int proximity_to_player;   /* 플레이어와의 거리 (상태 전이 조건)  */
      bool is_angry;/* 시간이 초과되어 속도가 빨라진 상태 */
} stENEMY; // 적 타입별은 이레님 위임

typedef struct { // 이레님 구현 위임
    // RANKING 점수 산출을 위한 데이터
    //
} stIN_GAME_MANAGER; // 인게임 상태에 대한 매니저

typedef struct {
    // LOADING을 위한 데이터들
    // MAIN LOOP
} stGAME_MANAGER;
```

---

7. 인터페이스 정의

기본 규칙

- 전역 변수 사용 금지
- 서로 작용은 함수를만들어서 포인터로 주고 받는 것
- const 쓰는 것
- 구조체 재사용
- 신규 구조체 생성 시 팀 합의

객체 간 인터페이스

```
void Collision(stCOLLISION *collision, stCOLLISION *collision);
-> 정원님께  위임
```

---

8. 역할 분담

역할 구조

```
Core (PM): 김진수
Player/Physics/Collision: 인정원
    - 모든 충돌 관련해서 상태 전이 정의 필요
Enemy/Game Logic: 최이레
    - 매프레임마다 오브젝트 상태 체크 필요
Rendering/Map/UI/Sound: 윤성민
    - State에 따라서 리소스 로딩 및 Sprite 출력 필요
```

상세 역할

■ PM / Core (김진수)

- Core
  - 메인 루프 구현
  - 입력 처리
  - 게임 상태 관리 (FSM)
  - 공통 구조 정의
  - 전체 통합
- PM
  - 프로젝트 일정 관리
  - 문서 작성 (계획서, 발표 자료)
  - 환경 구축 (Git, Notion, Visual Studio, Allegro)
  - 프로젝트 발표

■ Player / Physics / Collision (인정원)

- Player
  - Player FSM (IDLE / MOVE / JUMP / ATTACK / DEAD)
  - 이동 및 점프 구현
  - 공격 기능(Bubble)
- Physics / Collision
  - 중력 및 물리 처리
  - 충돌 처리
  - 사망 처리
  - Bubble 상호작용
  - 모든 상태 전이

■ Enemy / Game Logic (최이레)

- Enemy
  - 적 AI 구현
  - 적 FSM(IDLE/MOVE/HIT/DEAD)

    - 버블 됐을때 떠오르는 것(후순위)
- Game Logic
    - 점수 처리
    - 모든 오브젝트에 대해 탐색
    - 게임 규칙 구현

■ Rendering / Map / UI / Sound (윤성민)

- Rendering
    - Allegro 렌더링
    - Resource 로딩
    - Object FSM에 따른 이미지 렌더링
- Map
    - 타일맵 구현
- UI
    - HUD 구현
- Sound
    - BGM/피격 등에 따른 사운드 렌더링(후순위)

---

9. 프로젝트 구조

```
Source/
  main.c
  player.h
  player.c
  object.h
  physics.h
  physics.c
  collision.h
  collision.c
  bubble.h
  bubble.c
  enemy.h
  enemy.c
  ingame_manager.h
  ingame_manager.c
  game_manager.h
  game_manager.c
  render.h
  render.c
  map.h
  map.c

Assets/
  sprite/
```

```
map/
sound/
```

---

10. 개발 일정

3/25 (프로젝트 환경 구축; 오후까지 진행)

- 개발 환경 구축
  - Git → 리뷰 필요
  - Visual Studio
  - Notion
- Skeleton 실행 환경 완성 (공통)
  - 기본 구조체, 함수 정의
  - Allegro 초기화
  - Window 생성
  - Input 처리
  - 기본 게임 루프 구현
  - 인터페이스 정의

3/26 - 오전 (구현)

- 진수
  - 게임 전체 구조 잡기
    - 리소스 로딩
    - 인게임 전환
    - 스테이지 전환
- 성민
  - 맵(타일) 구현 -stage 1
  - 스프라이트 리소스 로딩
- 정원
  - 키 입력, 플레이어 Init, idle, move, 플레이어-맵 충돌
- 이레
  - Game logic 1차 구현
    - 점수 시스템 구현
    - 스테이지 시작/종료 전달 함수

3/26 - 오후 (구현)

- 진수
  - 코드 통합 및 조율
  - 성민님과 MAP 협업

- 성민
  - 오브젝트 FSM에 따라서 이미지 렌더링
- 정원
  - jump(일방향 발판), 중력, dead, 플레이어-적 충돌
- 이레
  - 적 FSM에 따른 동작 구현

3/27 - 오전 (구현)

- 진수
  - 코드 통합 및 조율
- 성민
  - stage 2, 3 MAP
- 정원
  - attack, bubble
- 이레
  - 적 AI, 투사체 구현

3/27 - 오후 (구현)

- 진수
  - 코드 통합 및 조율
- 성민
  - UI - 점수/메인/종료
- 정원
  - 투사체 - 플레이어, bubble-적 충돌
- 이레
  - 전체적인 Game logic 구현 및 디버깅

3/30 (발표 준비)

- PPT 작성 (PM)
- 영상 제작 (팀원)

3/31 (마무리)

- 최종 점검
- 버그 수정

---

11. Git 규칙

- 반드시 개인 branch에서 작업
- 작업 전 pull 필수

```
git pull origin main
```

- main 브랜치는 항상 실행 가능 상태 유지
- 강제 push 금지

```
git push -f ❌
```

---

12. 완료 기준

- 플레이어 이동 / 점프 / 공격 가능
- 적 정상 동작
- 충돌 처리 정상 작동
- 점수 시스템 구현
- 게임 전체 흐름 정상 작동

---

13. 리스크 및 대응

리스크

- 충돌 처리 난이도
- Allegro 초기 세팅 문제
- 인터페이스 불일치

대응

- AABB 기반 단순 충돌
- Day1 skeleton 확보
- 인터페이스 사전 정의

---

14. AI 사용 규칙

1. AI를 이용한 코드 구현 금지
2. 구현 방법 가능
3. 디버깅 가능